

# 统计学概论

杨文志  
安徽大学大数据与统计学院  
wzyang@ahu.edu.cn

## 第4章 数据结构与运算

- 数据类型
- 数据对象
- R函数与优化

## 4.1 数据类型

R语言的数据类型主要有**数值型**、**逻辑型**、**字符型**、**复数型**和**原始型**。

- 数值型 (numeric)：这种数据的形式是实数，可以写成整数 (integers)，小数 (decimal fractions)，或是科学记数 (scientific notation) 的方式。数值型实际上是两种独立模式的混合说法，即整数型 (integers) 和双精度型 (double-precision)，默认是双精度型数据。
- 字符型 (character)：这种数据的形式是夹在双引号" "或单引号' '之间的字符串，如"MR"。

## 4.1 数据类型

- 逻辑型 (logical)：这种数据只能取T (TRUE) 或F (FALSE) 值。
- 复数型 (complex)：这种数据是形如(a+bi)形式的复数。
- 原始型 (raw)：这种类型以二进制形式保存数据。
- 缺省值 (missing value)：有些统计资料是不完整的，当一个元素或值在统计的时候是“不可得到”或“缺失值”的时候，相关位置可能会被保留并且赋予一个特定的NA (not available) 值。任何NA的运算结果都是NA。is.na()用来检测数据是否缺失，如果数据缺失，则返回TRUE，否则，返回FALSE。

## 4.2 数据对象

R语言里的数据对象主要有六种结构：

向量 (vector)

矩阵 (matrix)

数组 (array)

因子 (factor)

列表 (list)

数据框 (data frame)

● **向量 (vector)** 是由有相同基本类型元素组成的序列，相当于一维数组。

- 函数c()用于生成向量，它可以有任意多个参数，输出的值是一个把这些参数首尾相连形成的一个向量。
- “#”符号后面跟的是注释。
- “<=”是赋值符号，表示把“<=”后面的内容赋值给“<=”前面的内容，而“=”赋值方法是把“=”后面的值赋给“=”前面的对象。对于字符向量，函数paste()函数可以把自变量对应元素连成一个字符串，长度不相同，较短的向量被重复使用。

**向量运算：**对于向量的乘法，除法，乘方运算，其方法是对应向量的每个分量做乘法、除法和乘方运算。

- 向量运算会对该向量的每一个元素都进行同样的运算。出现在同一个表达式的向量最好同一长度，如果长度不一，表达式中短的向量将会被循环使用，表达式的值将是一个和最长的向量等长的向量。

**向量常见函数：** 向量里元素的个数称为向量的长度（length），长度为1的向量就是常数（或标量）。

- 函数length()返回向量的长度，mode()返回向量的数据类型，min()和max()分别返回向量的最小值和最大值，range()返回向量的范围，which.min()和which.max()分别返回向量最小值和最大值的位置。
- R语言有很多内置函数，可以直接对向量进行运算，大大提高了工作效率。比如在R中用mean()来求向量的均值，median()求中位数，var()求方差，sd()求标准差等。

**向量索引：**

- R中提供了灵活的向量下标运算。取出向量x的第i个元素可以用x[i]得出。也可以通过赋值语句来改变一个或多个元素的值。
- 把i换成逻辑语句也可以对向量进行逻辑运算。
- 如果下标取值是负整数，则表示删除相应位置的元素。

**矩阵：** 矩阵是将数据用行和列排列的长方形表格，它是二维的数组，其单元必须是相同的数据类型。通常用列来表示不同的变量，用行表示各个对象。

**矩阵赋值**

R语言生成矩阵的函数是matrix()，其句法是：

matrix(data = NA, nrow = 1, ncol = 1, byrow = FALSE, dimnames = NULL)，其中，data 项为必要的矩阵元素，nrow为行数，ncol为列数，注意nrow与ncol的乘积应为矩阵元素个数，dimnames 给定行和列的名称，byrow项控制排列元素时是否按行进行，默认byrow=FALSE，即按列顺序排列。

**矩阵索引**

- 对于得到的矩阵，可以使用A[i,j]得到A矩阵第i行第j列的元素，A[i,]和A[,j]分别表示返回第i行和第j列的所有元素。
- 也可以使用A[i:k,j:l]的形式来获得多行多列的子矩阵。

## 矩阵运算

- 假定A为一个矩阵，则 的转置 $A^T$ 可以用函数t()来计算。
- 类似的，若将函数 t()作用于一个向量x，则当做x为列向量，返回结果为一个行向量。若想得到一个列向量,可用 t(t(x))。
- “\*”运算只是对应的元素相乘。矩阵的乘法在R中是使用“%\*%”来实现。用函数crossprod(A,B)还可以更高效地计算t(A)%\*%B。
- R中还可以对矩阵的对角元素进行计算。
- 另外，把diag()函数作用在一个向量上，可以产生以输入向量的元素为对角元的对角矩阵，如果输入的是一个正整数，会产生一个以该正整数为维度的单位阵。
- 在矩阵里面还有一个很重要的运算就是**求逆**，在R中使用solve()函数可以计算， solve(a,b)的返回值是线性方程组 $ax=b$ 的解，b默认为单位矩阵。

## 矩阵运算

- 对矩阵求特征值和特征向量的运算，在R中可以通过函数eigen()来实现。R中还可以对矩阵的对角元素进行计算。
- 在R中可以使用dim()得到矩阵的维数，nrow()求行数，ncol()求列数；rowSums()求各行和，rowMeans()求行均值，colSums()求各列和，colMeans()求列均值。
- row()和col()函数用来返回矩阵的行列下标。另外，也可以通过使用 $x[\text{row}(x)<\text{col}(x)]=0$ 等语句来得到矩阵的上下三角矩阵。对矩阵求特征值和特征向量的运算，在R中可以通过函数eigen()来实现。R中还可以对矩阵的对角元素进行计算。
- 对于矩阵的运算，我们还可以使用apply()函数来进行各种计算，其用法为： apply(X, MARGIN, FUN, ...), 其中，X表示需要处理的数据，MARGIN表示函数的作用范围，MARGIN=1时对行运算，MARGIN=2时对列运算。FUN为需要运用的函数。
- 矩阵合并还可以使用rbind()和cbind()函数来对矩阵按照行和列进行合并。

● **数组 (array)** 可以看作是带有多个下标的类型相同的元素的集合。也可以看作是向量和矩阵的推广，一维数组就是向量，二维数组就是矩阵。

- 数组的生成函数是`array()`，其句法是：`array(data = NA, dim = length(data), dimnames = NULL)`，其中`data`表示数据，可以为空，`dim`表示维数，`dimnames`可以更改数组的维度的名称。“#”符号后面跟的是注释。
- 数组`xx`是一个三维数组，其中第一维有三个水平，第二维有四个水平，第三维则有二个水平。索引数组类似于索引矩阵，索引向量可以利用下标位置来定义；其中`dim()`函数可以返回数组的维数。
- 此外，`dim()`还可以用来将向量转化成数组或矩阵。
- 数组也可以用“+”、“-”、“\*”、“/”以及函数等进行运算，其方法和矩阵相类似。

● **因子**：分类型数据 (category data) 经常要把数据分成不同的水平或因子 (factor)。因子代表变量的不同可能的水平（即使在数据中不出现）。

- 在R里可以使用`factor`函数来创建因子，函数形式如下：`factor(x = character(), levels, labels = levels, exclude = NA, ordered = is.ordered(x))`，其中，`levels`用来指定因子的水平；`labels`用来指定水平的名字；`exclude`表示在`x`中需要排除的水平；`ordered`用来决定因子的水平是否有次序。
- 要表示因子之间有大小顺序（考虑因子之间的顺序），则可以利用`ordered()`函数产生。

- **列表**：向量、矩阵和数组的元素都必须是同一类型的数据。 如果一个数据对象需要含有不同的数据类型，可以采用列表（list）。列表中包含的对象又称为它的分量（components），分量可以是不同的模式或类型，如一个列表可以包括数值向量、逻辑向量、矩阵、字符、数组等。

- 创建列表的函数是list()，其句法是：list（变量1=分量1， 变量2=分量2， ....）
- 若要访问列表的某一成分，可以用LST[[1]],LST[[2]]的形式访问，要访问第二个分量的前三个元素可以用LST[[2]][1:3]。
- 由于分量可以被命名，这时，我们可以在列表名称后加\$符号，再写上成分名称来访问列表分量。其中成分名可以简写到可以与其它成分能够区分的最短程度，如LST\$sc与LST\$score表示同样的分量。
- 在这里要注意LST[[1]]和LST[1]的差别，[[...]]是选择单个元素的操作符，而[...]是一个一般通用的下标操作符。因此前者得到的是LST中的第一个对象，并且包含分量名字的命名列表中的分量名字会被排除在外；而后者得到的则是LST中仅仅由第一个元素构成的子列表。

- **数据框**：是一种矩阵形式的数据，但数据框中各列可以是不同类型的数据。数据框每列是一个变量，每行是一个观测。数据框可以看成是矩阵（matrix）的推广，也可以看作是一种特殊的列表对象（list）。

- R语言中用函数data.frame ()生成数据框，其句法是： data.frame(..., row.names = NULL, check.rows = FALSE, ...)，数据框的列名默认为变量名，也可以对列名进行重新命名。
- 数据框是R语言特有的数据类型，也是进行统计分析最为有用的数据类型。不过对于可能列入数据框中的列表有如下一些**限制**：
  - 分量必须是向量（数值、字符或逻辑）、因子、数值矩阵、列表或者其他数据框。
  - 矩阵、列表和数据框为新的数据框提供了尽可能多的变量，因为它们各自拥有列、元素或者变量。
  - 数值向量、逻辑值、因子保持原有格式，而字符向量会被强制转换成因子并且它的水平就是向量中出现的独立值。
  - 在数据框中以变量形式出现的向量结构必须长度一致，矩阵结构必须有一样的行数。

## 数据框的引用

- **以数组形式访问数据框。**数组是储存数据的一种有效方法，可以按行或列访问，就像电子表格一样，但输入的数据必须是同一类型。数据框之所以可以看作数组是因为数据框的列表示变量、行表示样本观察数，因此我们可以访问指定的行或列。
- **以列表形式访问数据框。**列表比数据框更为一般、更为广泛。列表是对象的集合，而且这些对象可以是不同类型的。数据框是特殊的列表，数据框的列看作向量，而且要求是同一类型对象。可以以列表形式访问数据框，只要在列表名称后面加\$符号，再写上变量名就可以了。除了用\$形式访问外，还可以用列表名[[变量名(号)]]形式访问。

## 数据框的绑定

- 数据框的主要用途是保存统计建模的数据。R软件的统计建模功能都需要以数据框为输入数据。我们也可以把数据框当成一种矩阵来处理。在使用数据框的变量时可以用“数据框名\$变量名”的记法。但是，这样使用较麻烦，R软件提供了attach()函数可以把数据框中的变量“链接”到内存中，将数据框“连接（绑定）”入当前的名字空间，从而可以直接用变量名访问数据框中的变量。仍以数据框student为例，使用了attach(student)之后，就可以直接用score访问student中的score变量。
- 要取消连接，用函数detach()即可。

## 4.3 R函数与优化

条件控制语句

循环语句

编写自己的函数

程序调试

程序运行时间与效率

用R做优化求解

### 条件控制语句

- **if/else语句**

➤ if/else语句是分支语句中主要的语句，if/else语句的格式为

```
> if(cond) statement_1  
> if(cond) statement_1 else statement_2
```

➤ 第一句是指如果条件cond成立，则执行statement\_1，否则跳过。第二句是指如果条件cond成立，则执行statement\_1；否则执行statement\_2。

- **ifelse语句**

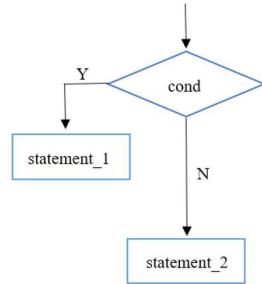
➤ ifelse 结构是if/else紧凑的向量化版本，其语法为：

```
> ifelse(cond,statement_1 else statement_2)
```

➤ 如果 cond 成立，则执行statement1，否则执行statement2。

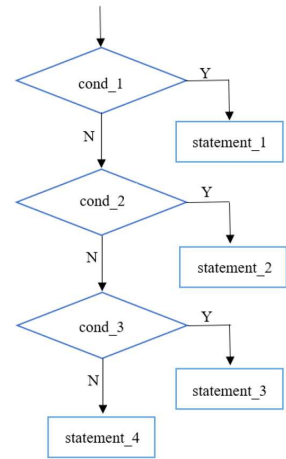
为帮助大家理解if/else语句程序化的表达，  
我们绘制了if/else语句对应的流程图

```
if (cond){
    statement_1
}else
    statement_2
}
```



更复杂的情形如下图：

```
> if (cond_1){
    statement_1
} else if (cond_2){
    statement_2
} else if (cond_3){
    statement_3
} else {
    statement_4
}
```



**【例4.1】** 例如我们想要计算当x = 4 时，下述两个分段函数的计算结果：

$$y = \begin{cases} 2+x, & x > 2 \\ 3x, & x \leq 2 \end{cases} \dots\dots(1) \quad , \quad y = \begin{cases} 2+x, & x > 3 \\ 3x, & 2 < x \leq 3 \dots\dots(2) \\ 4x, & x \leq 2 \end{cases}$$

```
> x<-4
> if(x>2) 2+x else 3*x
#假如x大于2，则返回2+x，否则
返回3*x
[1] 6
```

```
> x<-4
> ifelse(x>2, 2+x, 3*x)
#假如x大于2，则返回2+x，否则返
回3*x
[1] 6
```

```
> x<-4
> if(x>3){
    2+x
} else if(x>2){
    3*x
} else {
    4*x
}
#假如x大于3，返回2+x，假如x在2
和3之间，返回3*x，否则返回4*x
[1] 6
```

## ● switch语句

- switch语句是多分支语句，其使用方法为：

```
> switch(statement, list)
```

- 其中，statement是表达式，list是列表，可以用有名定义。如果表达式的返回值在1到length(list)，则返回列表相应位置的值。例如：

```
> switch(1,"beef","apple","potato") #返回"beef", "apple", "potato"中的第一个成分
[1] "beef"
> switch(2,"beef","apple","potato") #返回"beef", "apple", "potato"中的第二个成分
[1] "apple"
> switch(3,"beef","apple","potato") #返回"beef", "apple", "potato"中的第三个成分
[1] "potato"
```

- 当list是有名定义时，statement等于变量名时，返回变量名对应的值；否则，返回NULL值。例如：  

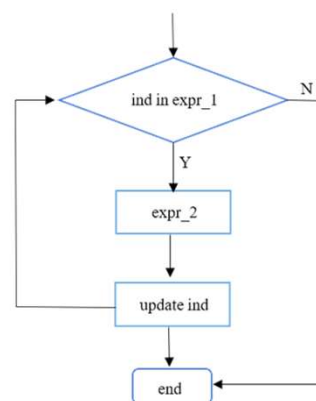
```
> x<-"fruit "
> switch(x, meat="beef", fruit="apple", vegetable="potato")
[1] "apple "
```

## 循环语句

### ● for循环

```
> for(ind in expr_1) expr_2
```

- 其中，ind是循环变量，expr\_1是一个向量表示式（通常是个序列，如-10:10），expr\_2通常是一组表达式。
- 其中，ind是循环变量，expr\_1是一个向量表示式（通常是个序列，如-10:10），expr\_2通常是一组表达式。同样，我们绘制for循环程序对应的流程图以帮助理解：



**【例4.2】** Fibonacci序列是数学中著名的序列，前两个元素都是1，第三个元素是第一、二个元素的和，第四个元素是第二、三个元素的和，一直下去。

$$a_1 = 1, a_2 = 1, a_n = a_{n-1} + a_{n-2}, n = 3, \dots$$

为了生成Fibonacci前14个元素，程序如下所示：

```
> Fibonacci<-NULL #生成一个空置向量
> Fibonacci[1]<-Fibonacci[2]<-1 # Fibonacci向量的第1和2个元素赋值为1
> n=14
> for (i in 3:n) Fibonacci[i]<-Fibonacci[i-2]+Fibonacci[i-1] #用for执行循环语句
> Fibonacci
[1] 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

### ● while循环

```
> while (condition) expr
```

- 只有当condition条件成立的时候，执行表达式expr。
- 例3-2: 编写小于500的Fibonacci序列：

```
> i<-1
> while (Fibonacci[i]+Fibonacci[i+1]<500) { #用while执行循环语句
  Fibonacci[i+2]<-Fibonacci[i]+Fibonacci[i+1]
  i<-i+1 }
> Fibonacci
[1] 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

## ● repeat循环语句

```
> repeat expr
```

➤ repeat循环依赖break语句跳出循环

➤ 例3-2: 编写小于500的Fibonacci序列:

```
> Fibonacci[1]<-Fibonacci[2]<-1 # Fibonacci向量的第1和2个元素赋值为1
> i<-1
> repeat { #用repeat执行循环语句
  Fibonacci[i+2]<-Fibonacci[i]+Fibonacci[i+1]
  i<-i+1
  if (Fibonacci[i]+Fibonacci[i+1]>=500) break
}
> Fibonacci
[1] 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

➤ 此处, break为中止语句。

## 自己编写的函数

R语言与其它统计软件最大的区别之一是你编写自己的函数, 而且可以像使用R的内置函数一样使用你的函数。

➤ 编写函数的句法是:

```
> 函数名= function (参数1, 参数2...)
{
  statements
  return(object)
}
```

➤ 函数名可以是任何值, 但以前定义过的要小心使用, 后来定义的函数会覆盖原先定义的函数。一旦你定义了函数名, 你就可以像R的其它函数一样使用, 比如我们定义一个求体重指数 (BMI指数) 的函数:  $BMI = \frac{w}{h^2}$

```
> BMI = function(w,h) { w/h^2 } #其中w表示体重 (kg), h表示身高 (m) }
```

- 对于这类只有一个语句的简单函数，也可不要{}，即

```
> BMI = function(w,h) w/h^{2}
```

```
> BMI(45,1.62) #计算体重为45kg，身高为1.62的人的BMI值
[1] 17.14678
```

- 假如我们不使用圆括号，直接输入函数名，按回车键将显示函数的定义式：

```
> BMI
function(w,h) w/h^{2}
```

### ● 关键词function

- 编写函数一定要写上function这个关键词，它告诉R这个新的数据对象是函数，所以编写函数时千万不可忘记。

### ● 参数

- 函数根据实际需要的不同而有不同的参数设置，下面将介绍三种情况：

**(1) 无参数：**有时编写函数是为了某种方便，函数每次的返回值都是一样的，其输入不是那么重要。比如我们编写“welcome”函数，其每次返回值都是“welcome to use R”。

```
> welcome = function() print("welcome to use R")
> welcome()
[1] "welcome to use R"
```

**(2) 单参数：**假如要使你的函数个性化，可以使用单参数，函数将会根据参数的不同，返回值也不同。

```
> welcome.ab = function(names) print(paste("welcome",names,"to use R"))
> welcome.ab("Mr fang")
[1] "welcome Mr fang to use R"
```

**(3) 多参数：**如果函数较为复杂，可以定义多个参数。比如我们编写一个模拟函数求服从均值为 $\mu$ ，标准差为 $\sigma$ 的正态样本数据的t统计量。这个函数的样本量、均值、标准差是可随意设置的。这时，我们就要在函数里添加样本量、均值、标准差三个参数。

```
> sim.t = function(n,mu,sigma){
  x=rnorm(n,mu,sigma)
  (mean(x)-mu)/(sd(x)/ sqrt(n))
}
> sim.t(50,0,1)      # 样本含量为50，均值为0，标准差为1
[1] 1.337119
> sim.t(100,sigma=10,mu=1) #样本含量为100，均值为1，标准差为10
[1] 0.4841053
```

- 这里值得注意的一点是，不要把位置参数与名义参数混淆起来。位置参数必须与函数定义的参数顺序一一对应。

**(4) 默认参数：**在仅设置参数名的情况下，调用函数时不输入任何参数的值会报错。但是在设置参数时，给参数设置默认值，调用函数时，不输入任何参数值的结果为将参数默认值代入函数的结果。以welcome.ab函数为例进行说明。

- 上面的welcome.ab函数假如不输入参数结果将会怎么样呢？

```
> welcome.ab()
错误在 paste("welcome", names, "to use R") :缺少变元 "names" ,也没有缺失值
```

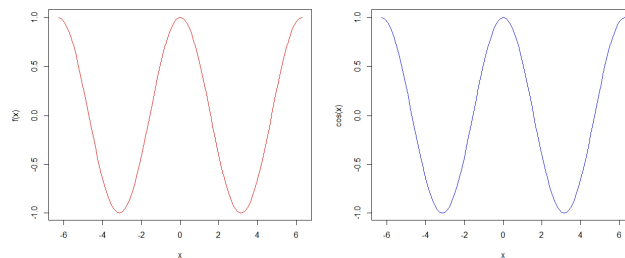
- 没有输入参数，函数welcome.ab将返回出错信息。其实我们可以给函数设置默认值，R提供了一个简单的方法允许给函数的参数设置默认值。比如：

```
> welcome.ab=function(names="Mr. Fang")print(paste("welcome", names,"to use R"))
> welcome.ab()
[1] "welcome Mr. Fang to use R"
```

- 除此之外，R语言允许定义一个变量，然后将变量值传递给R的内置函数。

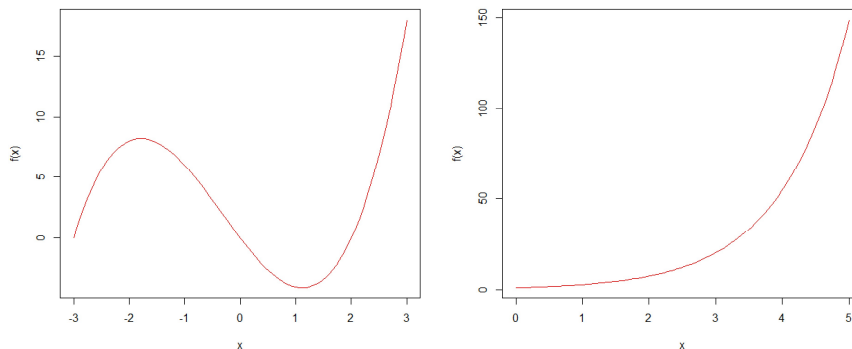
这在作图上非常有用。比如编写一个画图函数，允许你先定义一个变量 $x$ ，用这个变量生成 $y$ 变量，然后描出它们的图像。

```
> plot.f=function(f,a,b,...){
  xvals=seq(a,b,length=100) #生成100个[a,b]区间内的数列
  plot(xvals,f(xvals),type="l",col="red",xlab="x",ylab="f(x)",...) #作横坐标为x，纵坐标为f(x)的函数图
}
> par(mfrow=c(1,2)) #准备一张可以并排放置两张图片的画布
> plot.f(cos,-2*pi,2*pi) #作-2pi到2pi的余弦曲线，见下图左图
#这个函数的功能实际上等同于R内置函数curve。
> curve(cos,-2*pi,2*pi,col="blue") #作-2pi到2pi的余弦曲线，见下图
```



- 而由plot.f建立的函数是一种泛函， $f$ 可以是任何函数，例如指数函数、对数函数，也可以自定义函数。

```
> par(mfrow=c(1,2))
> f<-function(x){(x-2)*(x+3)*x} #自定义一个一元三次函数
> plot.f(f,-3,3) #绘制 -3 到3的一元三次函数图，见下图左图
> plot.f(exp,0,5) #绘制0到5的指数函数图，见下图右图
```



## ● 函数体和函数返回值

- 函数体和函数返回值是整个函数的主要部分，默认返回函数体的最后一个表达式的结果。如果函数体的表达式只有一个当然就很简单。例如：

```
> my.average = function(x) sum(x)/length(x)
> my.average(c(1,2,3))
[1] 2
```

- 当函数体的表达式超过一个时，要用{}封起来。R里也可以用return()返回函数需要的结果，当需要返回多个结果时，一般建议用list形式返回。例如我们用一个编写名为as()的函数来计算向量的均值和标准差。

```
> as<-function(x){
  a=mean(x)
  s=sd(x)
  return(list(avg=a,std=s))
}
> y<-c(5,15,32,25,26,28,65,48,3,37,45,54,23,44)
> as(y)
$avg
[1] 32.14286
$std
[1] 17.99084
```

- 编写程序，尤其是编写很长的程序是一件浩大的工程，费心费力，而且随着程序写的越长，越不好管理，也不好理解。因此，需要掌握以下几个技巧：
  - 建立从上到下设计的思想，将大的程序拆分成几块来写，每一块可以写成单独的函数。这有点像是盖楼，先把桩和框架搭好，然后再往里面逐步填充内容。
  - 将每一块又分成几步来写。
  - 应及时勤快地加注释。写好的程序过了几天，往往可能忘了其含义。
  - 尽可能做向量化运算。因为R将所有对象都存储在内存中，尽量少用循环(for,while)。用R内置函数 lapply, sapply, mapply等处理向量、矩阵或列表。
  - 在完整的数据集上运行程序前，先抽取部分数据子集进行测试，消除bug

## 程序调试

- 程序调试是每个程序员都头疼但是大多情况下必须面临的问题。计算机程序的错误或者缺陷叫“bugs”。调试程序一般分为如下几步：
  - 识别程序是否存在错误。有时很容易，因为如果程序出错，无法运行，那肯定是存在错误；但是有些时候，程序看似可行，结果却是错误的，或者对于有些输入是可行的，但对于有些输入不行，这类错误很难去识别。
  - 找出程序出错的原因。当程序有错，无法继续运行，R往往会给出报错信息，可以根据报错信息找出出错的原因。在R里还提供了traceback()、debug()、browser()函数可以跟踪和找出程序出错的地方。
  - 修正错误并测试。
  - 寻找类似的错误。

## 程序运行时间与效率

- R中proc.time()函数返回当前R已经运行的时间。例如

```
> proc.time()
 用户 系统 流逝
17.50 11.71 14840.82
```

其中“用户”是指R执行用户指令的CPU运行时间，“系统”是指系统所需的时间，“流逝”是指R打开到现在总共运行的时间。

- 计算程序运行时间的函数是：

```
> system.time(expr,gcFirst=TRUE)
```

其中expr是需要运行的表达式，gcFirst是逻辑参数。system.time是实际上两次调用了proc.time()，在程序运行前调用一次，运行完后调用一次，然后计算两次的时间差，即为程序的运行时长。

```
> system.time(for(i in 1:100) mad(rnorm(1000))) #mad求中位数
 用户 系统 流逝
0.01 0.00 0.02
```

- 该程序等价于：

```
> ptm<-proc.time() #将proc.time()的返回值保存到ptm对象里
> for(i in 1:100) mad(rnorm(1000))
> proc.time()-ptm #两者的差即运行程序所需的时间
 用户 系统 流逝
0.03 0.00 0.03
```

➤ R设计主要是基于向量和矩阵的运算。比如求两个向量的差：

• 程序1：

```
> ptm<-proc.time()
> n<- 1000000
> x<-runif(n)
> y<-runif (n)
> z<-c()
> for (i in 1:n){
  z<-c(z,x[i]-y[i])
}
> proc.time()-ptm
  用户  系统  流逝
1631.56 1.07 1661.16
```

➤ 该程序写得非常没有效率，总共运用了1661.16秒时间。程序每次都增加一个元素到Z向量里，这样总共增加了1000000次。如果我们既然已经知道了Z的长度，我们首先就给定Z的长度，这样可以提高运行效率。

• 程序2：

```
> ptm<-proc.time()
> n<- 1000000
> x<-runif(n)
> y<-runif (n)
> z<-rep(NA,n)
> for (i in 1:n){
  z[i]<-x[i]-y[i]
}
> proc.time()-ptm
  用户  系统  流逝
  0.23  0.02  0.25
```

➤ 总共运行了0.25秒，比程序1效率高了很多，运行效率相当于是程序1的6644倍。最后，我们直接使用矩阵运算符进行运算，可以进一步提高效率。

• 程序3：

```
> ptm<-proc.time()
> n<- 1000000
> x<-runif(n)
> y<-runif (n)
> z<-x-y
> proc.time()-ptm
  用户  系统  流逝
  0.07  0.00  0.06
```

➤ 该程序只要运行0.06秒就可以了。

➤ 从左边例子可以看出R中循环算法的效率极低，当我们不得不书写循环算法时，可以考虑借助R的并行包提高计算速度。可以实现并行运算的R包有parallel、snowfall等。以下述程序为例。

```
> n<-8^{8}
> ptm<-proc.time()
> k<-lapply(1:n, function(x) {if(x>500) x+2 else x+4})
> proc.time()-ptm
  用户  系统  流逝
16.01  8.63 28.91
```

➤ 该程序对大于500的数加上2，小于等于500的数加上4。在不进行并行运算时，总共运行28.91秒。

```

> n<-8^{8}
> library(parallel)
> ptm<-proc.time()
> cores<-4 #设置线程，线程越多，速度越快
> ori<-makeCluster(cores) #初始化
> K<-parLapply(ori,1:n, function(x) {if(x>500) x+2 else x+4})
> stopCluster(ori) #结束并行
> proc.time()-ptm
  用户   系统  流逝
  4.36   1.22  18.58

```

- 当使用并行运算时，总共运行了18.58秒，和不进行并行运算相比，效率得到了提升。

## 用R做优化求解

优化是在统计计算中非常重要的一部分内容，因为很多统计方法，比如最小二乘法和极大似然估计法，归根到底就是求目标函数的最小值或者最大值。接下来，先讲解最简单的一元函数的优化求解，然后再讲解多元函数的优化求解。

### ● 一元函数优化求解

- R做一元的最优化求解函数是

```

> optimize(f, interval = ..., lower = min(interval), upper = max(interval),
           maximum = FALSE, tol = .Machine$double.eps^0.25)

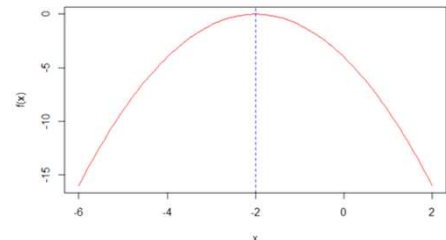
```

- 其中，f是需要求优化的函数，interval是参数搜索的区间，lower是参数搜索的下限，如果缺失，用interval的最小值代替，upper是参数搜索的上限，缺失时用interval的最大值代替，maximum默认是FALSE，表示求极小值，tol是精度的容忍度。

### 用R做优化求解

- 例4.3 求解 $f(x) = -x^2 - 4x - 4$ ，这是二次函数的图形如图所示。该函数只有一个极大值，利用拉格朗日方法可以求得在 $x = -2$ 处取得最大值。

```
> f<-function(x){-x*x-4*x-4} #编写f函数
> optimize(f,c(-6,2),tol=0.0001,maximum=T) #求f函数的
  最大值的返回结果
$maximum
[1] -2
$ objective
[1] 0
```



- 利用R求解出来的当 $x = -2$ 目标函数的最大值为0，这与利用拉格朗日方法求解的结果相同。

### 用R做优化求解

#### ● 多元函数优化求解

- 多元函数的优化求解可以使用函数`optim()`求解，其用法如下

```
> optim (par, fn, gr = NULL, ...,
        method = c("Nelder-Mead", "BFGS", "CG", "L-BFGS-B", "SANN"),
        lower = -Inf, upper = Inf, control = list(), hessian = FALSE)
```

- 其中`par`设定初始值，`fn`是需要优化的目标函数，`gr`是梯度向量，如果是`NULL`则由`optim()`计算所得的近似值替代。`lower`是参数搜索的下限，默认是 $-\infty$ ，`upper`是参数搜索的上限，默认是 $+\infty$ 。`control`是用来控制`optim`函数的一些参数，`hessian=FALSE`表示不需要返回海塞矩阵。在计算机里，优化的求解实质是通过迭代算法所得。`optim()`提供的迭代算法主要有Nelder-Mead, BFGS, CG, L-BFGS-B, SANN。

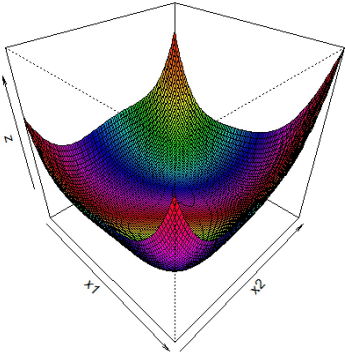
用R做优化求解

➤ 例3-5：求解两元函数 $g(x_1, x_2) = (x_1^2 + x_2 + 1)^2 + (x_1 + x_2^2 - 7)^2$ 的极值。

对该两元函数的优化求解：

➤ Step1：用R写出目标函数表达式，并画出该函数的三维图，直观上分析其极值。

```
> x1=x2=seq(-10,10,length=100)
> fr1=function(x1,x2){
  (x1^2+x2+1)^2+(x1+x2^2-7)^2
}
> z=outer(x1,x2,fr1) #求外积
> persp(x1,x2,z,box=T,border=T,theta=45,phi=35,col="
blue") #绘制三维图
```



➤ 所得的三维图如右图所示。

用R做优化求解

➤ Step2：设置优化算法迭代初始值，利用optim()进行优化求解。

```
> fr2=function(x){
  x1<-x[1]
  x2<-x[2]
  (x1^2+x2+1)^2+(x1+x2^2-7)^2
}
> grr<-function(x){
  x1<-x[1]
  x2<-x[2]
  c(4*(x1^2+x2+1)*x1+2*(x1+x2^2-7),2*(x1^2+x2+1)+4*(x1+x2^2-7)*x2)
} #一阶导数
> optim(c(-1,-1),fr2,grr) #设初始值为-1, -1，不同初始值的优化结果可能
不同
$par
[1] 1.187770 -2.410791
$value
[1] 1.016734e-07
$counts
  function gradient
    73      NA
$convergence
[1] 0
$message
NULL
```

➤ 需要注意的是不同的初始值的优化结果可能不同，将上述问题几种不同初始值的优化结果整理成如下所示。从下表可以看出，该两元函数应该有四个局部最小值。

表 不同初始值下的优化结果

| 初始值        | 参数最终值          | 极小目标值                  |
|------------|----------------|------------------------|
| $c(-1,-1)$ | (1.188,-2.411) | $1.017 \times 10^{-7}$ |
| $c(-2,-3)$ | (-1.376,2.894) | $8.451 \times 10^{-4}$ |
| $c(3,-2)$  | (1.187,-2.411) | $7.396 \times 10^{-8}$ |
| $c(-4,3)$  | (1.188,-2.411) | $9.204 \times 10^{-7}$ |

## 用R做优化求解

### ● 约束条件下的优化求解

- 有时求解目标函数的极值是在一定的约束条件下进行。例如，求解两元函数  $g(x_1, x_2) = (x_1^2 + x_2 + 1)^2 + (x_1 + x_2^2 - 7)^2$  的极值。此时的目标函数是在  $x_1 > 0, x_2 > 0$  的约束下达到最小值。对于约束下的最优化问题，有两种方法可以求解：
- 一种方法是利用 `constrOptim()` 函数直接求解，该函数的用法：

```
> constrOptim(theta, f, grad, ui, ci, mu = 1e-04, control = list(),
  method = if(is.null(grad)) "Nelder-Mead" else "BFGS", hessian = FALSE)
```

`theta`是初始值向量，`f`是目标函数，`grad`是梯度向量，可以是空值NULL，`ui`是约束矩阵的左边系数矩阵，`ci`是约束矩阵的右边的值。

## 用R做优化求解

- 比如，  $g(x_1, x_2) = (x_1^2 + x_2 + 1)^2 + (x_1 + x_2^2 - 7)^2$

- 对于上面的两元函数，其约束条件是  $x_1 > 0, x_2 > 0$ ，等价于  $\begin{cases} 1x_1 + 0x_2 > 0 \\ 0x_1 + 1x_2 > 0 \end{cases}$ ，所

以  $ui = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, ci = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ 。

```
> uimat=rbind(c(1,0),c(0,1))
> cimat=c(0,0)
> constrOptim(c(0.2,0.5),fr2,grr,ui=uimat,ci=cimat)#其中fr2为目标函数，grr为一阶导数
$par
[1] 0.1004318 2.4893149
$value
[1] 12.73985
$counts
  function gradient 
        30       12 
$convergence
[1] 0
$message
NULL
$outer.iterations
[1] 3
$barrier.value
[1] 5.502996e-05
```

➤ `par`是目标函数达到极值时的  $x_1$  和  $x_2$  的取值分别为 0.1004318, 2.4893149; `value`是目标函数的极值，为12.73985; `$counts` 表示迭代过程中用到目标函数（`function`）30次，梯度函数（`gradient`）12次; `convergence`取0表示成功收敛。